Name: Anjan Singh

Index

S.No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	12-Aug-25	Full adder and Subtractor using Gate		
3	26-Aug-25	Decoder using Test Bench		
4	28-Aug-25	Multiplexer And Demultiplexer Code in verilog		

(*) Aim :- To design simulate and verify the Magnification functionality of various fundamental combinational logic circuits using Verilog HDL.

(*) Theory :-

This experiment covers the design and verification of standard combinational circuits and fundamental concepts of the Verilog language.

(*) Tools required :-

(1) Model Sim :- This is an HDL simulation tool used for verifying the functionality of digital circuits.

(2) Verilog HDL :- This is the language used to describe the digital circuits and their best benches.

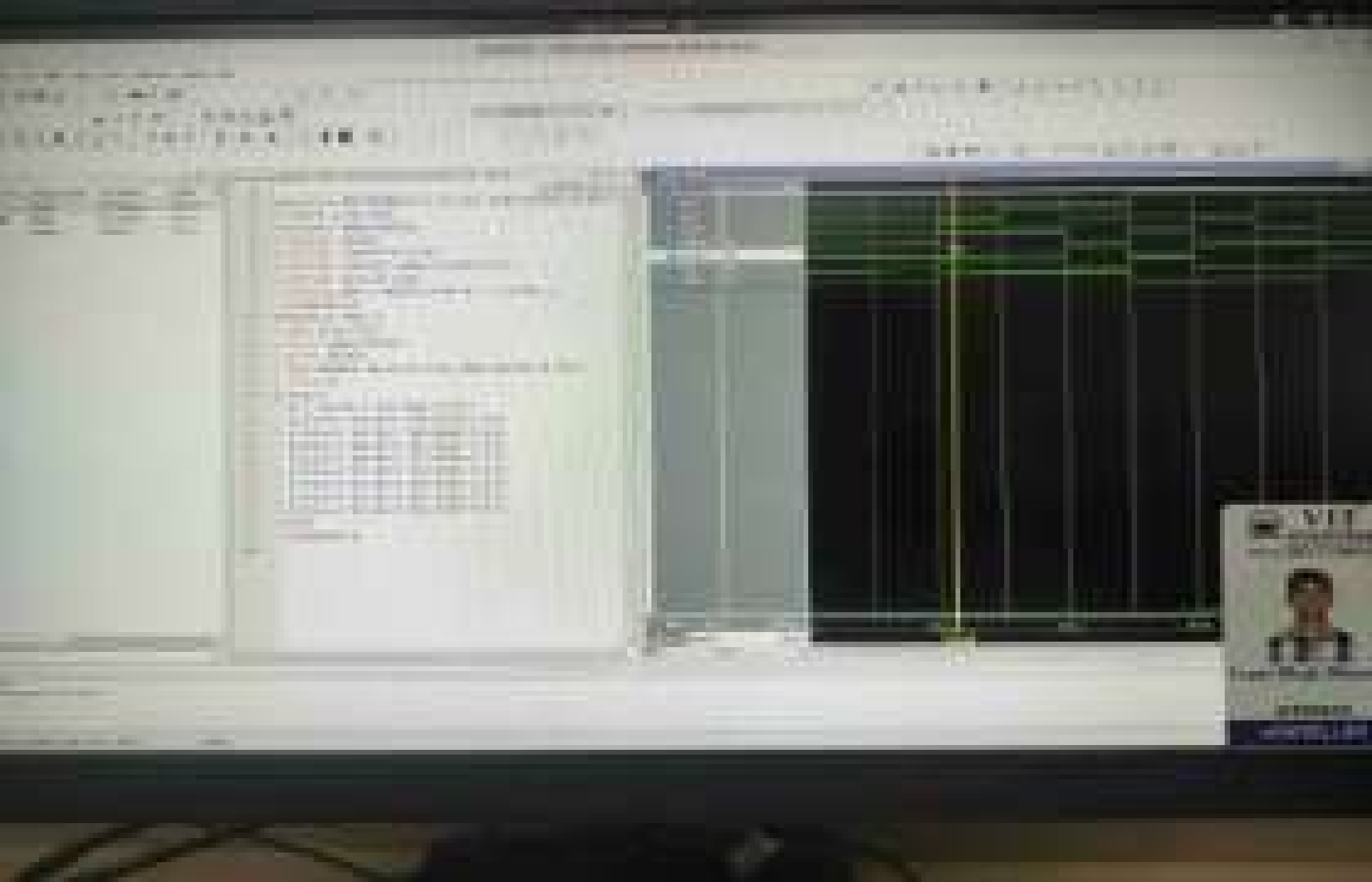
Date 21-Aug-25

{-Assesment - 2}

Full Adder

```
module full_adder (a, b, cin, sum, carry,d,b0);
input a, b, cin;
output sum, carry;
output d, b0;
assign sum = a^b^cin;
assign carry = (a&b)|(cin&(a^b));
assign d = a^b^cin;
assign b0 = ((~a)&b)|((a^b)&(~cin));
endmodule
module tb();
reg a, b, cin;
wire sum, carry;
wire d, b0;
full_adder m0 (a, b, cin, sum, carry, d, b0);
initial
begin
a=1'b0; b=1'b0; cin=1'b0;
#100 a=1'b0; b=1'b0; cin=1'b1;
#100 a=1'b0; b=1'b1; cin=1'b0;
#100 a=1'b0; b=1'b1; cin=1'b1;
#100 a=1'b1; b=1'b0; cin=1'b0;
#100 a=1'b1; b=1'b0; cin=1'b1;
#100 a=1'b1; b=1'b1; cin=1'b0;
#100 a=1'b1; b=1'b1; cin=1'b1;
end
endmodule
```


VIT
VELLORE CAMPUS
Aryan Singh Bikarwa
HOSTELLER



A	B	C	D	Bo	
T	T	T	T	T	
T	T	F	F	F	
T	F	T	F	F	
T	F	F	T	F	
F	T	T	F	T	
F	T	F	T	T	
F	F	T	T	T	
F	F	F	F	F	

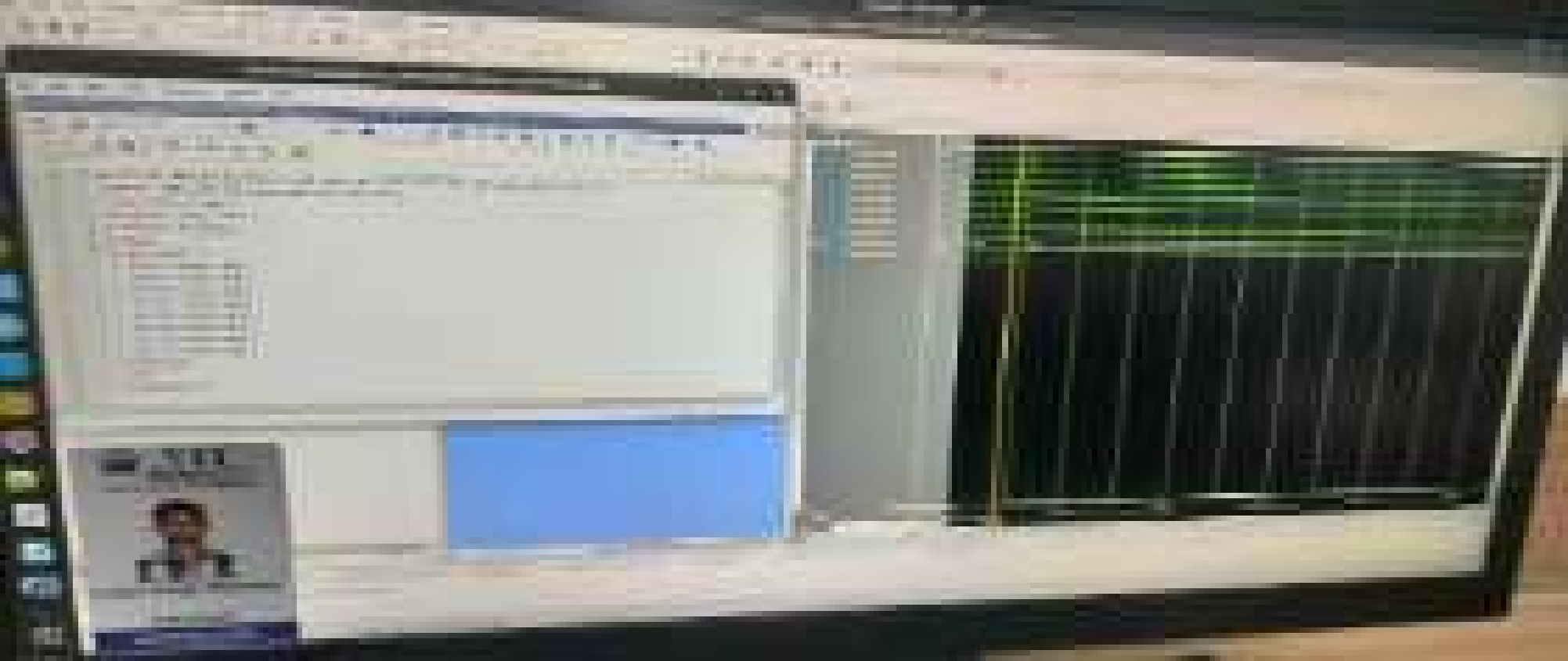
#2 Decoder Test Bench code :-

```
module decoder (A,B,e,do,d1,d2,d3);
input A,B;
output do,d1,d2,d3;
assign do = ~A & ~B & e;
assign d1 = ~A & B & e;
assign d2 = A & ~B & e;
assign d3 = A & B & e;
endmodule tb();
reg A,B,e;
wire do,d1,d2,d3;
decoder dec (A,B,e,do,d1,d2,d3);
Initial
begin
A=1'b0; B=1'b0; e=1'b1;
#100 A=1'b0; B=1'b0; e=1'b1;
#100 A=1'b1; B=1'b0; e=1'b1;
#100 A=1'b1; B=1'b1; e=1'b1;
end
endmodule
```

Multiplexer 8:1

```
module Multiplexer (d0, d1, d2, d3, d4, d5, d6, d7, sel, out);
input d0, d1, d2, d3, d4, d5, d6, d7;
input [2:0] sel;
output reg out;
always @ (sel)
begin
case (sel)
3'b000 : out d0;
3'b001 : out d1;
3'b010 : out d2;
3'b011 : out d3;
3'b100 : out d4;
3'b101 : out d5;
3'b110 : out d6;
3'b111 : out d7;
end
endmodule
```

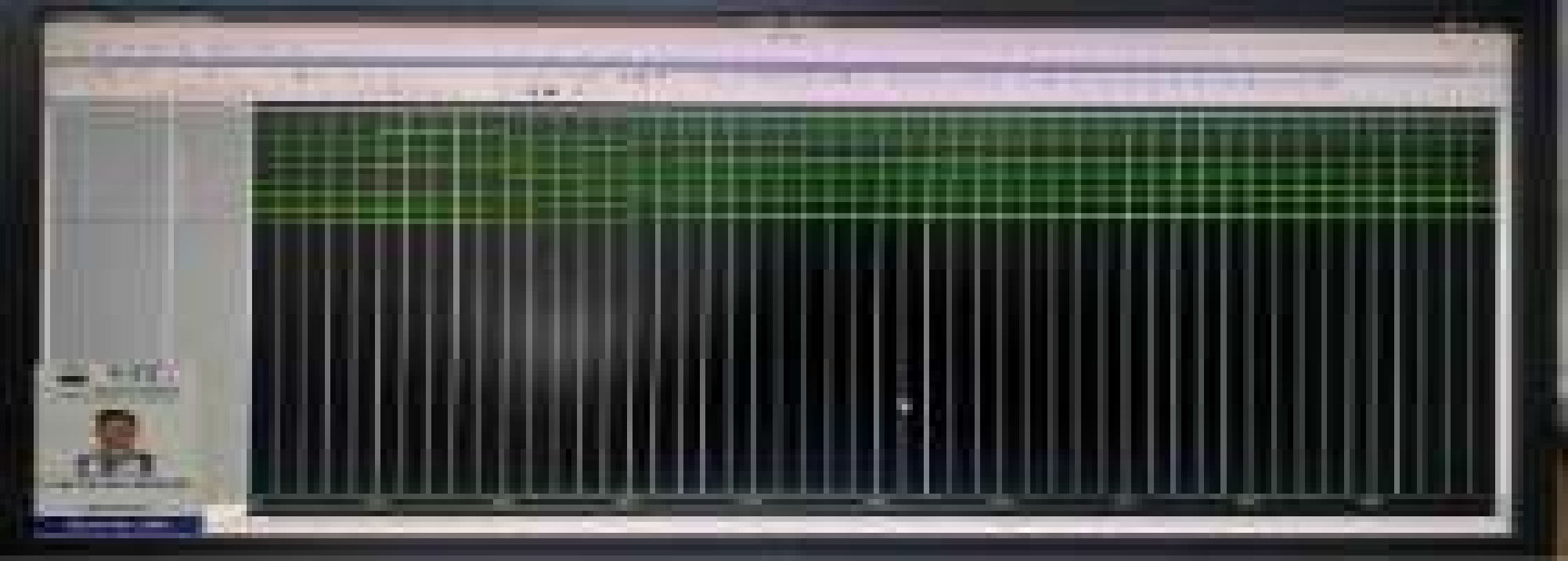
```
module Testbench;
reg d0, d1, d2, d3, d4, d5, d6, d7;
reg [2:0] sel;
wire out;
multiplexer uut (.d0(d0), .d1(d1), .d2(d2), .d3(d3), .d4(d4), .d5(d5), .d6(d6), .d7(d7), .sel(sel), .out(out));
initial begin
d0=0; d1=0; d2=0; d3=0; d4=0; d5=0; d6=0; d7=0; sel=0;
#100;
d0=0; d1=0; d2=0; d3=0; d4=1; d5=1; d6=0; d7=1;
```

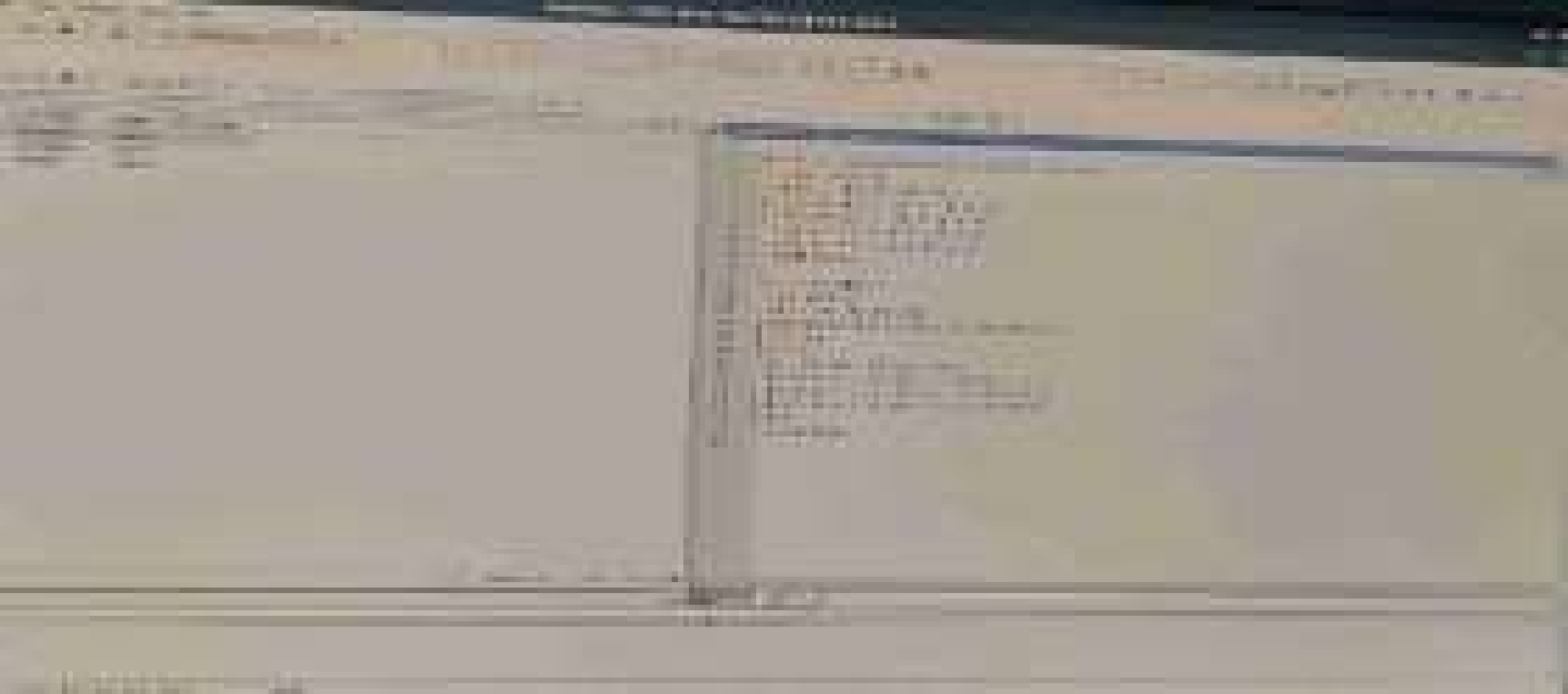


Demultiplexer Code :-

```
module demultiplexer (a,x,y,z,d0,d1,d2,
  d3,d4,d5,d6,d7);
input a,x,y,z;
output d0,d1,d2,d3,d4,d5,d6,d7;
assign d0 = (a&~x&~y&~z);
d1 = (a&~x&~y&z);
d2 = (a&~x&y&~z);
d3 = (a&~x&y&z);
d4 = (a&x&~y&~z);
d5 = (a&x&~y&z);
d6 = (a&x&y&~z);
d7 = (a&x&y&z);
endmodule

module Testbench_demux;
reg a,x,y,z;
wire d0,d1,d2,d3,d4,d5,d6,d7;
demultiplexer out(.a(a),.x(x),.y(y),
.z(z),.d0(d0),.d1(d1),.d2(d2),.d3(d3),
.d4(d4),.d5(d5),.d6(d6),.d7(d7));
initial
begin
a=0; x=0; y=0; z=0;
#100 a=1; x=0; y=1; z=0;
#100 a=1; x=1; y=0; z=0;
#100 a=1; x=1; y=1; z=1;
end
endmodule
```





Answer the following

* what do you mean by test Bench in verilog?

Ans It creates a copy of the module you want to test and places it inside the virtual lab. It seprates piece of a code you write to check if your main hardware design.

* what is Difference between initial and always?

Ans The initial block runs only once at the beginning of a simulation typically for setting up a test. The always block runs repeatedly and continuously, modeling the behaviour of real hardware always active.

* what is instalation in verilog?

Ans Instantiation is the process of creating a usable copy of one module inside another. It is the fundamental method for building complex, hierarchical circuits from smaller, reusable